
Ejercicio 1: El Dilema de la Taquilla (Exclusión Mutua con Mutex)

Descripción del Problema

Contexto: Una cadena de cines tiene una base de datos centralizada que lleva el conteo de boletos vendidos para una sala con capacidad limitada.

El Conflicto: Durante un estreno, miles de usuarios acceden simultáneamente desde la App y la página web. Si dos procesos leen que queda "1 asiento disponible" al mismo tiempo, ambos procederán a venderlo, resultando en una sobreventa (condición de carrera).

Objetivo: Implementar un mecanismo de Exclusión Mutua (Mutex) que actúe como un "cerrojo" en la taquilla, asegurando que solo un hilo pueda procesar una venta a la vez en la sección crítica.

Ejercicio 2: El Gimnasio (Semáforos de Conteo)

Descripción del Problema

Contexto: Un gimnasio tiene 3 máquinas de press de banca. Hay 8 atletas que quieren entrenar al mismo tiempo.

El Conflicto: Como no hay máquinas para todos, los atletas que llegan y encuentran las máquinas ocupadas deben esperar en una fila.

Objetivo: Implementar un Semáforo General (de conteo) que gestione el acceso a las máquinas. El semáforo comenzará con el valor 3. Cada vez que alguien usa una máquina, el semáforo baja; cuando termina, sube. Si el semáforo llega a 0, el siguiente atleta se bloquea automáticamente hasta que el semáforo sea mayor a 0.

Ejercicio 3: La Vitrina de la Panadería (Productor-Consumidor)

Descripción del Problema

Contexto: Una panadería tiene una vitrina con capacidad limitada (Buffer).

El Conflicto:

1. El Panadero no puede poner más pan si la vitrina está llena (debe esperar a que alguien compre).
2. El Cliente no puede comprar pan si la vitrina está vacía (debe esperar a que el panadero hornee).
3. Solo una persona puede tocar la vitrina a la vez (Mutex) para evitar que un pan se caiga o se registre mal.

Objetivo: Gestionar la sincronización usando dos semáforos de conteo (espacios_vacios y panes_listos) y un cerrojo de exclusión mutua.

Ejercicio 4: El Tablón de Notas (Lectores-Escritores)

Descripción del Problema

Contexto: Un recurso compartido (el tablón) debe ser accedido por dos tipos de usuarios.

El Conflicto:

1. Lectura Concurrente: Varios estudiantes pueden leer a la vez (no se estorban).
2. Escritura Exclusiva: Solo un profesor puede escribir a la vez, y mientras escribe, nadie puede leer (para evitar ver datos incompletos o erróneos).
3. Inanición (Starvation): Si siempre hay estudiantes leyendo, el profesor nunca podrá actualizar las notas. Esto es el problema de Prioridad a Lectores.

Objetivo: Implementar la lógica donde el primer lector "cierra la puerta" a los escritores y el último lector "la abre".

Ejercicio 5: El Punto de Encuentro (Barreras de Sincronización)

Descripción del Problema

Contexto: Un sistema de computación paralela donde varios procesos realizan una tarea en etapas (Fase 1 y Fase 2).

El Conflicto: La Fase 2 requiere que todos los datos de la Fase 1 estén listos. Si un hilo rápido comienza la Fase 2 antes de que los lentos terminen la Fase 1, los datos estarán incompletos y el sistema fallará.

Objetivo: Implementar una Barrera usando variables de condición que detenga a los hilos "rápidos" hasta que el contador de hilos llegue al total esperado

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

EJERCICIO 1: ALGORITMO

- Inicializar un contador de boletos_vendidos en 0.
- Definir un objeto Mutex (cerrojo).
- Crear N hilos (clientes/servidores de venta).
- Cada hilo debe:
 - ✓ Intentar adquirir el Mutex.
 - ✓ Sección Crítica: Leer el valor actual, incrementarlo y guardarlo.
 - ✓ Liberar el Mutex.
- Esperar a que todos los hilos terminen y verificar que el total sea exactamente el esperado (N X M)

ALGORITMO VentaAtómica

VARIABLES

entero_atómico boletos_vendidos <- 0

```
constante N_HILOS <- 5
constante M_VENTAS <- 1000000
```

```
PROCEDIMIENTO EjecutarVenta()
  PARA i DESDE 1 HASTA M_VENTAS HACER
    // La operación INCREMENTAR_ATÓMICO es indivisible
    INCREMENTAR_ATÓMICO(boletos_vendidos)
  FIN PARA
FIN PROCEDIMIENTO

INICIO
  PARA j DESDE 1 HASTA N_HILOS HACER
    INICIAR_HILO(EjecutarVenta)
  FIN PARA

  ESPERAR_HILOS()
  MOSTRAR "Ventas Totales: ", boletos_vendidos
FIN
```

EXPLICACION

Si un hilo llega a la instrucción BLOQUEAR y el mutex ya está tomado por otro, el hilo actual se queda "dormido" (en espera) hasta que el otro ejecute DESBLOQUEAR.

EJERCICIO 2: ALGORITMO

Inicializar un Semaforo con el valor de los recursos disponibles (3).

Para cada Atleta (Hilo):

- ✓ Solicitar: Ejecutar wait() sobre el semáforo. Si el valor es > 0, decrementa y pasa. Si es 0, espera.
- ✓ Sección Crítica: Usar la máquina por un tiempo aleatorio.
- ✓ Liberar: Ejecutar signal() o release() sobre el semáforo para avisar que la máquina está libre.

ALGORITMO ControlGimnasio

CLASE MiSemaforo

VARIABLES

contador: ENTERO

cerrojo: MUTEX

cola_espera: CONDICION

CONSTRUCTOR(valor)

contador <- valor

METODO Esperar()

```
BLOQUEAR(cerrojo)
MIENTRAS contador == 0 HACER
    ESPERAR_CONDICION(cola_espera, cerrojo)
FIN MIENTRAS
contador <- contador - 1
DESBLOQUEAR(cerrojo)
FIN METODO
```

```
METODO Senial()
    BLOQUEAR(cerrojo)
    contador <- contador + 1
    NOTIFICAR(cola_espera)
    DESBLOQUEAR(cerrojo)
FIN METODO
FIN CLASE
```

Explicación

1. Operación Esperar () (El intento de entrada)

Este es el protocolo que sigue un proceso cuando quiere usar una máquina:

- ✓ Protección de Datos: El hilo primero adquiere un Mutex. Esto es vital porque si dos hilos intentan restar 1 al contador al mismo tiempo, el contador se corrompería.
- ✓ La Verificación (El bucle): El hilo mira el contador.
 - Si el contador es 0: El hilo ejecuta un wait(). Esto hace dos cosas mágicas: suelta el Mutex (para que otros puedan salir) y se queda dormido en una fila especial.
 - Si el contador es > 0: Salta al paso 3.
- ✓ Toma del Recurso: El hilo resta 1 al contador (contador = contador - 1).
- ✓ Liberación del Mutex: El hilo suelta el cerrojo y se va a usar su máquina.

2. Operación Senial () (La salida)

Este es el protocolo cuando el proceso termina:

- ✓ Protección de Datos: Adquiere el Mutex para modificar el contador de forma segura.
- ✓ Incremento: Suma 1 al contador (contador = contador + 1). Ahora hay una máquina libre.
- ✓ La Notificación: El hilo ejecuta un notify() o signal(). Esto envía una "señal eléctrica" a la sala de espera.
 - Si había hilos dormidos, uno de ellos se despierta.
 - El hilo que despierta vuelve a intentar el paso 2 del acquire().
- ✓ Liberación del Mutex: El hilo sale del sistema.

EJERCICIO 3: ALGORITMO

Lógica de los "Vigilantes"

- El sistema se basa en tres guardias de seguridad:
- Semáforo Vacío: Empieza en 10. Indica cuántos huecos quedan.
- Semáforo Lleno: Empieza en 0. Indica cuántos panes hay para comer.
- Mutex: El permiso para abrir la puerta de la vitrina.

ALGORITMO Panaderia

VARIABLES

```
espacios_vacios <- SEMAFORO(10)
panes_listos <- SEMAFORO(0)
mutex_vitrina <- MUTEX(1)
vitrina <- ARRAY[10]
```

PROCEDIMIENTO Panadero()

```
MIENTRAS (quede jornada laboral) HACER
    pan <- Hornear()
```

```
    ESPERAR(espacios_vacios) // ¿Cabe en la vitrina?
    BLOQUEAR(mutex_vitrina) // Abrir vitrina
```

```
    Vitrina.insertar(pan)
```

```
    DESBLOQUEAR(mutex_vitrina) // Cerrar vitrina
    SEÑAL(panes_listos) // Avisar al cliente
```

```
FIN MIENTRAS
```

```
FIN PROCEDIMIENTO
```

PROCEDIMIENTO Cliente()

```
MIENTRAS (tenga hambre) HACER
```

```
    ESPERAR(panes_listos) // ¿Hay pan disponible?
    BLOQUEAR(mutex_vitrina) // Abrir vitrina
```

```
    pan <- Vitrina.sacar()
```

```
    DESBLOQUEAR(mutex_vitrina) // Cerrar vitrina
    SEÑAL(espacios_vacios) // Avisar al panadero
```

```
    Comer(pan)
```

```
FIN MIENTRAS
```

```
FIN PROCEDIMIENTO
```

```
FIN ALGORITMO
```

Explicación

Semaforos: el panadero y el cliente miran cosas distintas. El panadero mira el espacio vacío; el cliente mira el pan cocinado.

Mutex: Para que el Panadero no esté poniendo el pan justo en el mismo lugar de donde el Cliente lo está agarrando, lo que podría "romper" el pan (corrupción de datos).

El Buffer Circular: Como la vitrina tiene fin, cuando llegamos al espacio 10, el siguiente pan vuelve al espacio 0. Es como una rueda que nunca deja de girar.

EJERCICIO 4: ALGORITMO

- Contador de Lectores: Una variable para saber cuántos estudiantes hay frente al tablón.
- Mutex de Lectores: Para que el conteo de estudiantes sea exacto.
- Semáforo de Escritura: Es la "llave del tablón".
 - ✓ Si un estudiante llega y es el primero, intenta tomar la llave. Si el profesor la tiene, espera.
 - ✓ Si llegan más estudiantes, pasan directo (porque el primero ya aseguró la llave).
 - ✓ Cuando un estudiante se va y es el último, devuelve la llave.

ALGORITMO TablonNotas

VARIABLES

```
entero cant_lectores <- 0
mutex mutex_lectores <- 1
semaforo sem_escritor <- 1
```

PROCEDIMIENTO Lector()

```
// --- SECCIÓN DE ENTRADA ---
BLOQUEAR(mutex_lectores)
cant_lectores <- cant_lectores + 1
SI cant_lectores == 1 ENTONCES
    ESPERAR(sem_escritor) // Si soy el primero, pido la llave
FIN SI
DESBLOQUEAR(mutex_lectores)
```

```
Leer_Tablon() // SECCIÓN CRÍTICA (Compartida entre lectores)
```

```
// --- SECCIÓN DE SALIDA ---
```

```
BLOQUEAR(mutex_lectores)
cant_lectores <- cant_lectores - 1
SI cant_lectores == 0 ENTONCES
    SEÑAL(sem_escritor) // Si soy el último, devuelvo la llave
FIN SI
```

DESBLOQUEAR(mutex_lectores)

FIN PROCEDIMIENTO

PROCEDIMIENTO Escritor()

ESPERAR(sem_escritor) // El escritor siempre pide la llave exclusiva

Escribir_Tablon() // SECCIÓN CRÍTICA (Exclusiva)

SEÑAL(sem_escritor)

FIN PROCEDIMIENTO

FIN ALGORITMO

Explicación

1. El Proceso del Lector (Estudiante)

El lector ahora tiene que pasar por un control adicional antes de acercarse al tablón.

- ✓ Paso A: El estudiante llega y mira si el semáforo orden_llegada está libre. Si un profesor lo tiene, el estudiante se queda quieto ahí mismo.
- ✓ Paso B: Si el estudiante logra pasar, y nota que es el primero en llegar, va y pone el seguro en el tablón (llave_escritura) para que ningún profesor entre mientras él y sus amigos leen.
- ✓ Paso C: Muy importante: el estudiante suelta el orden_llegada rápido para que otros compañeros puedan entrar a leer detrás de él.

2. El Proceso del Escritor (Profesor)

Aquí es donde ocurre la magia de la prioridad. El profesor no espera a que el tablón esté vacío para "hacer fila".

- ✓ Paso D: En cuanto el profesor llega, toma el semáforo orden_llegada. A diferencia del lector, el profesor NO lo suelta de inmediato. Esto hace que cualquier estudiante que llegue después se quede bloqueado en el "Paso A" del código anterior.
- ✓ Paso E: Ahora que ya nadie nuevo puede entrar, el profesor solo tiene que esperar a que los estudiantes que ya estaban leyendo terminen y salgan.
- ✓ Paso F: Una vez que el último estudiante sale y libera la llave_escritura, el profesor entra, toma la llave, suelta el orden_llegada (para que otros profesores hagan fila detrás) y escribe.

EJERCICIO 5: ALGORITMO (LÓGICA DE LA "LLAMADA DE LISTA")

- Contador: Lleva el registro de cuántos hilos han llegado a la barrera.
- Mutex: Protege el contador para que dos hilos no se registren al mismo tiempo.
- Variable de Condición (barrier): Es el megáfono que mantiene a todos en silencio hasta que el guía da la orden.
- Broadcast: La acción de avisar a todos los hilos en espera simultáneamente.

ALGORITMO BarreraSincronizacion

VARIABLES

entero contador <- 0 // Cuenta cuántos procesos han llegado

```
constante N_TOTAL <- 5      // Meta de hilos necesarios
mutex mtx_barrera           // Protege el acceso al contador
condicion var_cond_barrera   // Donde los hilos esperan
```

```
PROCEDIMIENTO LlegarABarrera(id_hilo)
```

```
  BLOQUEAR(mtx_barrera)      // Paso 1: Poner candado
  contador <- contador + 1
```

```
  SI contador == N_TOTAL ENTONCES
```

```
    // Paso 2: El último hilo despierta a todos
```

```
    NOTIFICAR_A TODOS(var_cond_barrera)
```

```
  SINO
```

```
    // Paso 3: Los demás se duermen y sueltan el mutex temporalmente
```

```
    MIENTRAS contador < N_TOTAL HACER
```

```
      ESPERAR_EN_CONDICION(var_cond_barrera, mtx_barrera)
```

```
    FIN MIENTRAS
```

```
  FIN SI
```

```
  DESBLOQUEAR(mtx_barrera)   // Paso 4: Quitar candado al salir
```

```
FIN PROCEDIMIENTO
```

```
FIN ALGORITMO
```

Explicación

A. ¿Por qué el Mutex (mtx_barrera)?

Si 5 hilos intentan hacer `contador <- contador + 1` al mismo tiempo sin un candado, podrías terminar con un error de hardware donde el contador marca "3" aunque ya llegaron los 5. El Mutex asegura que la fila para anotarse sea de uno en uno.

B. El "Efecto Dominó" (NOTIFICAR_A TODOS)

Cuando el hilo número 5 llega, no se duerme. Él ejecuta la instrucción de "Despertar a todos". Esto es como si el guía del bus gritara por el megáfono: "¡Ya estamos todos, despierten!".

C. El bucle MIENTRAS (Seguridad)

No usamos un SI para esperar, sino un MIENTRAS. Esto es por si un hilo se despierta por error (un "despertar falso" del sistema operativo). Al despertar, el hilo vuelve a mirar el contador: si sigue siendo menor a N_TOTAL, se vuelve a dormir inmediatamente.

D. ESPERAR

Cuando un hilo ejecuta ESPERAR_EN_CONDICION, ocurre algo especial: el hilo suelta el mutex automáticamente antes de dormirse. Si no lo soltara, el siguiente hilo nunca podría entrar a la función para sumar su llegada al contador, y el sistema quedaría bloqueado para siempre.

6. Resultados esperados:

- Comprobación de las distintas formas de como los hilos se comunican.

- Se espera que el estudiante mida y compare el rendimiento entre las soluciones secuenciales y concurrentes
- El estudiante debe demostrar comprensión de los conceptos vistos en cada ejercicio de la practica

Ejercicio	Resultado Esperado	Criterio de Verificacion
1. Mutex	Counter siempre = $N \times M$ (5,000,000)	Ejecutar 10 veces sin variacion
2. Semaforos	resources_in_use nunca supera 3	Monitorear valor maximo alcanzado
3. Prod-Cons	Todos los items producidos son consumidos	Verificar secuencia y completitud
4. Lect-Esc	Lectores concurrentes, escritor exclusivo	Log muestra patron de acceso correcto
5. Barrera	Todos los hilos esperan antes de fase 2	Ningun hilo inicia fase 2 antes

7. Preguntas de Control:

- ¿Qué es una condición de carrera?
- Cuál es la diferencia fundamental entre un semáforo binario y un mutex? ¿Pueden usarse indistintamente? Justifique su respuesta con al menos dos diferencias prácticas
- Explique qué es un interbloqueo (deadlock) y enuncie las cuatro condiciones necesarias de Coffman. ¿Cuál de estas condiciones se elimina al usar la estrategia de ordenamiento de recursos?

8. Evaluación

Criterio	Excelente	Bueno	Bajo
Funcionamiento (back-end, front-end) se evaluara criterios de Verificacion	Funciona completo (4)	Parcial (2)	No funciona (0)
Documento de practica (Enlace a git, metodología usada por el alumno, resultados, preguntas de control, bibliografía,	Funciona completo (3)	Parcial (2)	No funciona (0)